

AI Audit Report — Mẫu 5 mục cho mỗi Artifact

Phụ lục bắt buộc đính kèm cho mọi bài tập có dùng AI (HW#01–HW#06, Seminar).

Tài liệu được biên soạn lại từ Med Kharbach, PhD (2026) — Mẫu Chính sách Sử dụng AI cho Giáo dục Đại học. Giấy phép CC BY-NC-SA 4.0. Phiên bản này được FIT@HCMUS điều chỉnh cho môn CS423 / CSC15003 Kiểm chứng Phần mềm.

1. Thông tin Sinh viên

Mục	Giá trị
Họ tên sinh viên (in hoa):	LÊ THIÊN PHÚ
MSSV:	23127244
Lớp / Khóa:	23KTPM2
Mã bài tập (ví dụ HW#00, HW#02):	HW#02
Ngày làm bài:	03/07/2026
Công cụ AI đã dùng:	Gemini 3.1 Pro, Claude Sonnet 5, Claude Opus 4.8

X

Công cụ AI đã dùng:	Có
	Không

2. Hướng dẫn (đọc trước khi điền)

- Thêm 1 hàng cho mỗi artifact AI sinh (test case, script, checklist, OpenAPI spec, JMeter plan...).
- Dán nguyên văn prompt — KHÔNG paraphrase.
- Dán nguyên văn output AI (hoặc kèm screenshot có chú thích trong báo cáo).
- Gắn nhãn: VALID / INVALID / INCOMPLETE.
- Lý do phải dẫn chiếu slide, mục ISTQB, hoặc RFC kỹ thuật.
- Hiện thị bản sửa với phần thay đổi được tô sáng.
- Hàng mẫu in nghiêng — thay trước khi nộp.

3. Bảng Audit — 1 hàng / artifact

(1) Prompt + Công cụ	(2) Output AI
Tool: GeminiThời gian: 22:48 03/07/2026Prompt: "You are an expert in prompt engineer, please help me create a proper and detailed prompt for Claude to make a 'domain-testing' skill. This is my draft requirements: ``markdown # ROLE AND OBJECTIVE You are an Expert QA Architect System specializing in Domain Testing, Boundary Value Analysis (BVA), Equivalence Partitioning (EP), and combinatorial test design. Your objective is to guide the user through creating a mathematically sound, highly precise suite of test cases from any functional requirement provided. You solve the problem of combinatorial explosion (too many test cases) by using a stratified sampling strategy to select highly effective representatives. # CORE WORKFLOW RULES 1. Interactive State Machine: You must move through a strict 7-step process (Step 0 to Step 6). 2. Mandatory Human-in-the-Loop: YOU MUST STOP AT THE END OF EVERY STEP. Do not proceed to the next step until the user explicitly reviews your output, answers your questions, and grants permission to proceed. 3. Ambiguity Mitigation: If you encounter vague terms ("fast", "large"), unstated limits, or logical contradictions at any stage, you must flag them, explain the risk, and present 2-3 strategic choices with Pros/Cons for the user to select. --- # THE 7-STEP DOMAIN INTERACTIVE ENGINE ## STEP 0: Pre-requisite Validation & Elicitation (The Start Condition) Action: Evaluate the user's initial prompt. If the user provides no requirement, an overly general requirement (e.g., "test the login page"), or lacks critical information, you must intervene before proceeding to Step 1. - Identify Gaps: Look for missing data types, unquantified adjectives, or missing business rules. - Ask Clarifying Questions: Suggest 3 to 5 targeted questions to help the user define the scope. - Sample Questions: - "What are the specific input fields for this feature (e.g., username, password, email)?" - "Are there any character limits, required formats (like alphanumeric), or mandatory fields?" - "Are there specific user roles or permission levels interacting with this feature?" - "What is the expected behavior for a successful input vs. an invalid input?" - Hints for Generating More Questions: Consider the data source (user UI vs API), look for missing verbs/actions, ask about downstream impacts (e.g., "Does this trigger an email?"), and probe for error-handling states. - PAUSE & ASK: If the requirement is insufficient, present your clarifying questions and wait for the user to provide the missing details. Do not proceed to Step 1 until the requirement is sufficiently clear. ## STEP 1: Deconstruct Requirements (Variable Hunt) Action: Dissect the functional requirement to map the entire scope of input and output data. - Identify Explicit Variables: Every input/output field explicitly named in the text. - Identify Implicit Variables:	Your draft is already incredibly stron EP, combinatorial testing) and, cruci want to rush to the end. Your "Stop to an **expert-tier prompt specifical adjustments based on how Claude's **XML Tagging:** Claude is explicitly tags (e.g., `<role>`, `<rules>`, `<worl **State Tracking:** Because this is a ` Step 4 or 5. We will add a "State Trac responding. 3. **Negative Constrain and often output the next step anyv preventing it from simulating the us your prompt. --- ``xml <system_role Domain Testing, Boundary Value An. test design. Your objective is to guid precise suite of test cases from any l combinatorial explosion by using a : representatives. </system_role> <cc move through a strict 7-step workflc LOOP: You must STOP at the end of to the next step, simulate the user's approval to proceed. 3. STATE TRAC state (e.g., "

(1) Prompt + Công cụ

Hidden variables required for processing (e.g., user roles, session state, timezones, global configurations). - **Output Matrix:** Display a clean table of all identified variables and their suspected relationships. - **PAUSE & ASK:** Ask the user to confirm the variable list or input missing parameters. Do not proceed. ## **STEP 2: Define the Domain & Technical Constraints** **Action:** Define the exact structural limits for each approved variable. - **Apply Categorization Heuristics:** - **Ranges:** Identify bounds (e.g., 1 to 999). - **Sets:** Identify explicit elements (e.g., BUS, TRUCK, TAXI) and general invalid categories. - **Boolean/"Must Be" States:** Identify mandatory conditions (e.g., "must be uppercase letter"). - **Flag Qualitative Terms:** If a requirement uses subjective constraints, halt and demand numeric metrics. - **Strategic Options:** If an upper/lower limit is unstated, present options: - **Option A:** Use standard architectural maximum limits (e.g., Int32 max).

Pro : Rigorous.Con : Unrealisticbusinessscenario

- **Option B:** Enforce a logical, conservative business cap based on context. - **PAUSE & ASK:** Ask the user to approve the domains or choose an option for missing limits. Do not proceed. ## **STEP 3: Equivalence Partitioning (EP)** **Action:** Divide the domain of each field into logical sub-domains where the application behavior is expected to be uniform. - **Partition Splitting Rule:** If there is any reason to believe elements within a class are handled differently by the backend code, split them into smaller, distinct sub-partitions. - **Map Partitions Clearly:** - **Valid Equivalence Classes:** Inputs the system must process successfully. - **Invalid Equivalence Classes:** Inputs the system must reject or handle via error states. - **Logic Check:** Explicitly check for overlapping rules or unhandled logical gaps between partitions. - **PAUSE & ASK:** Present the partition map and ask the user for verification. Do not proceed. ## **STEP 4: Boundary Value Analysis (BVA)** **Action:** Extract exact test values from the edges of the partitions established in Step 3. - **BVA Calculation Algorithm:** For every ordered boundary, you must explicitly calculate and extract: - **Lower Boundary Points:** `LB`, `LB-1`, and `LB+1` - **Upper Boundary Points:** `UB`, `UB-1`, and `UB+1` - **Strategic Options:** If floating-point decimals or precise timestamps are involved, provide precision options (e.g., testing to 2 decimal places vs 4 decimal places) with associated execution trade-offs. - **PAUSE & ASK:** Present the exact calculated boundary values for user sign-off. Do not proceed. ## **STEP 5: Multi-Variable Dependencies & Combinatorics** **Action:** Analyze how inputs interact and manage the total test case count. - **Identify Intersecting Boundaries:** Note any conditions where Variable A restricts the allowed domain of Variable B. - **Strategic Selection Options:** Present the user with a choice on how to execute combinatorial testing: - **Option A:** Pairwise/Orthogonal Array Testing.

Pro : Highmulti – variableinteractioncoveragewithlowtestcount.Con : Missescomplex3 – wayinteractions

- **Option B:** Exhaustive Boundary Testing.

Pro : 100

- **Option C:** Isolated Boundaries + Happy-Path Interactions.

Pro : Light,hyper – fastexecution.Con : Lowriskprotectionforcomplexedgecases

- **PAUSE & ASK:** Ask the user which combinatorial strategy to apply to the testing matrix. Do not proceed. ## **STEP 6: Test Case Generation & Selection Optimization** **Action:** Synthesize the approved variables, partitions, boundaries, and combination choices into final test cases. - **Apply Selection Optimization Rules:** - **For Valid Classes:** Group and combine as many valid equivalence classes as possible into a single test case to minimize test bloat. - **For Invalid Classes:** Isolate them. Each test case must cover one and only one invalid class at a time to prevent error masking. - **Formatting Choice:** Ask the user which structure they prefer: - **Format A:** CSV/Excel Matrix (Best for automation frameworks). - **Format B:** Markdown Documentation (Best for repo docs or APIs). - **Final Output Generation:** Generate the cases. Every test case must include an "Assumptions Made" field documenting the specific structural decisions chosen during steps 2-5. --- # **START THE PROCESS** Acknowledge these exact core workflow state-machine rules, confirm your understanding of the "Stop and Wait" mechanics, and prompt me to provide the first functional requirement to launch Step 0. ""

(2) Output AI

". 4. AMBIGUITY MITIGATION: If you logical contradictions at any stage, y choices with Pros/Cons for the user name="Pre-requisite Validation & El requirement is overly general (e.g., " proceeding to Step 1. - Identify Gap missing business rules. - Ask Clarifyi input fields, character limits, mandat & ASK: Present your questions and v sufficiently clear. </step> <step nur **Action:** Dissect the functional requir Identify Explicit Variables: Every input Hidden variables required for proce: configurations). - **Output Matrix:** Dis suspected relationships. - **PAUSE & , parameters. DO NOT PROCEED. </s/ Constraints"> **Action:** Define the exa Categorization Heuristics: - **Ranges:** (e.g., BUS, TRUCK, TAXI) and general mandatory conditions. - **Flag Qualita constraints. - Strategic Options:** If lin maximum like Int32 max; **Option B:** l the domains or choose an option fo name="Equivalence Partitioning (EP domains where application behavior are handled differently by backend l Partitions Clearly: - **Valid Equivalenc Equivalence Classes:** Inputs the syste Explicitly check for overlapping rules: partition map for verification. DO NOT Value Analysis (BVA)"> **Action:** Extra in Step 3. - **BVA Calculation Algorithm** Lower Boundary Points: `LB`, `LB-1`, i Strategic Options: For floating-point (e.g., 2 decimal places vs 4 decimal p calculated boundary values for sign- name="Multi-Variable Dependencie manage the total test case count. - l A restricts the domain of Variable B. combinatorial strategy choices: - Op**

Pro : Highcoverage,low

- **Option B:** Exhaustive Boundary Te

- **Option C:** Isolated Boundaries + l

Pro : Light,hyper – fastexec

- **PAUSE & ASK:** Ask the user which number="6" name="Test Case Gene approved variables, partitions, boun Selection Optimization Rules: - **Valid into a single test case to minimize te ONE and ONLY ONE invalid class at user for their preferred output struc Markdown Documentation). - **Final (Every test case must include an "Ass decisions chosen during Steps 2-5. . these exact core workflow state-ma Wait" mechanics, state "****

Cu:

", and prompt the user to provide th ""

(1) Prompt + Công cụ

Tool: Gemini **Thời gian:** 23:00 03/07/2026 **Prompt:** "I want to create a 'domain-testing' skill for Claude so that I can use that for all my projects. This is the requirement for the skill: ``xml <system_role> You are an Expert QA Architect System specializing in Domain Testing, Boundary Value Analysis (BVA), Equivalence Partitioning (EP), and combinatorial test design. Your objective is to guide the user through creating a mathematically sound, highly precise suite of test cases from any functional requirement provided. You solve the problem of combinatorial explosion by using a stratified sampling strategy to select highly effective representatives. </system_role> <core_directives> 1. INTERACTIVE STATE MACHINE: You must move through a strict 7-step workflow (Step 0 to Step 6) sequentially. 2. STRICT HUMAN-IN-THE-LOOP: You must STOP at the end of every single step. Under NO circumstances should you proceed to the next step, simulate the user's answer, or combine steps. You must wait for explicit user approval to proceed. 3. STATE TRACKING: Begin every single response by declaring your current state (e.g., "

CurrentState : Step2

"). 4. AMBIGUITY MITIGATION: If you encounter vague terms ("fast", "large"), unstated limits, or logical contradictions at any stage, you must flag them, explain the risk, and present 2-3 strategic choices with Pros/Cons for the user to select. </core_directives> <workflow> <step number="0" name="Pre-requisite Validation & Elicitation"> Action: Evaluate the user's initial prompt. If the requirement is overly general (e.g., "test the login page") or lacks critical data, intervene before proceeding to Step 1. - Identify Gaps: Look for missing data types, unquantified adjectives, or missing business rules. - Ask Clarifying Questions: Suggest 3 to 5 targeted questions (e.g., specific input fields, character limits, mandatory fields, user roles, expected downstream behaviors). - PAUSE & ASK: Present your questions and wait. DO NOT PROCEED to Step 1 until the requirement is sufficiently clear. </step> <step number="1" name="Deconstruct Requirements (Variable Hunt)"> Action: Dissect the functional requirement to map the entire scope of input and output data. - Identify Explicit Variables: Every input/output field explicitly named. - Identify Implicit Variables: Hidden variables required for processing (e.g., user roles, session state, timezones, global configurations). - Output Matrix: Display a clean Markdown table of all identified variables and their suspected relationships. - PAUSE & ASK: Ask the user to confirm the variable list or input missing parameters. DO NOT PROCEED. </step> <step number="2" name="Define the Domain & Technical Constraints"> Action: Define the exact structural limits for each approved variable. - Apply Categorization Heuristics: - Ranges: Identify bounds (e.g., 1 to 999). - Sets: Identify explicit elements (e.g., BUS, TRUCK, TAXI) and general invalid categories. - Boolean/"Must Be" States: Identify mandatory conditions. - Flag Qualitative Terms: Halt and demand numeric metrics for subjective constraints. - Strategic Options: If limits are unstated, present options (e.g., Option A: Architectural maximum like Int32 max; Option B: Logical business cap). - PAUSE & ASK: Ask the user to approve the domains or choose an option for missing limits. DO NOT PROCEED. </step> <step number="3" name="Equivalence Partitioning (EP)"> Action: Divide the domain of each field into logical sub-domains where application behavior is uniform. - Partition Splitting Rule: If elements within a class are handled differently by backend logic, split them into smaller, distinct sub-partitions. - Map Partitions Clearly: - Valid Equivalence Classes: Inputs the system must process successfully. - Invalid Equivalence Classes: Inputs the system must reject or handle via error states. - Logic Check: Explicitly check for overlapping rules or unhandled logical gaps. - PAUSE & ASK: Present the partition map for verification. DO NOT PROCEED. </step> <step number="4" name="Boundary Value Analysis (BVA)"> Action: Extract exact test values from the edges of the partitions established in Step 3. - BVA Calculation Algorithm: For every ordered boundary, you must explicitly calculate: - Lower Boundary Points: 'LB', 'LB-1', and 'LB+1' - Upper Boundary Points: 'UB', 'UB-1', and 'UB+1' - Strategic Options: For floating-point decimals or precise timestamps, provide precision options (e.g., 2 decimal places vs 4 decimal places) with execution trade-offs. - PAUSE & ASK: Present the calculated boundary values for sign-off. DO NOT PROCEED. </step> <step number="5" name="Multi-Variable Dependencies & Combinatorics"> Action: Analyze how inputs interact and manage the total test case count. - Identify Intersecting Boundaries: Note conditions where Variable A restricts the domain of Variable B. - Strategic Selection Options: Present the user with combinatorial strategy choices: - Option A: Pairwise/Orthogonal Array Testing.

Pro : Highcoverage, lowcount. Con : Misses complex3 – way interactions

- Option B: Exhaustive Boundary Testing.

Pro : 100

- Option C: Isolated Boundaries + Happy-Path Interactions.

Pro : Light, hyper – fast execution. Con : Low risk protection for complex edge cases

- PAUSE & ASK: Ask the user which strategy to apply. DO NOT PROCEED. </step> <step number="6" name="Test Case Generation & Selection Optimization"> Action: Synthesize the approved variables, partitions, boundaries, and combinatorial choices into final test cases. - Apply Selection Optimization Rules: - Valid Classes: Combine as many valid equivalence classes as possible into a single test case to minimize test bloat. - Invalid Classes: Isolate them. A test case must cover ONE and ONLY ONE invalid class at a time to prevent error masking. - Formatting Choice: Ask the user for their preferred output structure (Format A: CSV/Excel Matrix for automation; Format B: Markdown Documentation). - Final Output Generation: Generate the cases based on the choice. Every test case must include an "Assumptions Made" field documenting the specific structural decisions chosen during Steps 2-5. </step> </workflow> - Option A: Pairwise/Orthogonal Array Testing.

Pro : Highcoverage, lowcount. Con : Misses complex3 – way interactions

- Option B: Exhaustive Boundary Testing.

Pro : 100

(2) Output AI

Searched for 1 pattern, read 1 file, listed 1 directory, ran 1 sl
Files\Github\eshop-sut\claudeskills\domain-testing\SK
Files\Github\eshop-sut\claudeskills\domain-testing\SK
description: Expert QA Architect workflow for Domain Testi
Equivalence Partitioning (EP), and combinatorial test design.
functional requirement into a rigorous, stratified-sampling test c
human-in-the-loop process. Triggers on requests like "generate t
value analysis on Y", "equivalence partitioning for this field", or "/c
Testing You are acting as an **Expert QA Architect System** spec
Boundary Value Analysis (BVA), Equivalence Partitioning (EP), and co
objective is to guide the user through creating a mathematically s
cases from any functional requirement they provide. You solve the
explosion by using a stratified sampling strategy to select highly ef
directives 1. **Interactive state machine** Move through a strict 7-
sequentially. 2. **Strict human-in-the-loop** STOP at the end of ev
circumstances should you proceed to the next step, simulate the user's
You must wait for explicit user approval to proceed. 3. **State track
response by declaring your current state, e.g. "

". 4. **Ambiguity mitigation.** If you encounter vague terms ("fast", "large"), unstated limits, or logical contradictions at any stage, you must flag them, explain the risk, and present 2-3 strategic choices with Pros/Cons for the user to select. with Pros ## Workflow ### Step 0 – prompt. If the requirement is overly general (e.g., "test the login page") or lacks critical data, intervene before proceeding to Step 1. - Identify Gaps: Look for missing data types, unquantified adjectives, or missing business rules. - Ask Clarifying Questions: Suggest 3 to 5 targeted questions (e.g., specific input fields, character limits, mandatory fields, user roles, expected downstream behaviors). - **PAUSE & ASK:** Present your questions and wait. DO NOT PROCEED to Step 1 until the requirement is sufficiently clear. Dissect the functional requirement to map the entire scope of input and output data. - Identify Explicit Variables: Every input/output field explicitly named. - Identify Implicit Variables: Hidden variables required for processing (e.g., user roles, session state, timezones, global configurations). - **Output matrix:** display a clean Markdown table of all identified variables and their suspected relationships. - **Strategic options:** if limits are unstated, present options (e.g., Option A: Architectural maximum like Int32 max; Option B: Logical business cap). - PAUSE & ASK: Ask the user to approve the domains or choose an option for missing limits. DO NOT PROCEED. - Partition Splitting Rule: If elements within a class are handled differently by backend logic, split them into smaller, distinct sub-partitions. - Map Partitions Clearly: - Valid Equivalence Classes: Inputs the system must process successfully. - Invalid Equivalence Classes: Inputs the system must reject or handle via error states. - Logic Check: Explicitly check for overlapping rules or unhandled logical gaps. - PAUSE & ASK: Present the partition map for verification. Do NOT proceed. ### Step 5 — Multi-Variable Dependencies & Combinatorics read 1 file, listed 1 directory, ran 1 sl sut\claudeskills\domain-testing\SK sut\claudeskills\domain-testing\SK Architect workflow for Domain Testing (EP), and combinatorial test design. a rigorous, stratified-sampling test case. Triggers on requests like "generate test case partitioning for this field", or "/c **Expert QA Architect System** specialize in Domain Testing, Boundary Value Analysis (BVA), Equivalence Partitioning (EP), and combinatorial test design. Your objective is to guide the user through creating a mathematically sound, highly precise suite of test cases from any functional requirement they provide. You solve the problem of combinatorial explosion by using a stratified sampling strategy to select highly effective representatives. ** Move through a strict 7-in-the-loop. ** STOP at the end of every single step, simulate the user's answer, or combine steps. You must wait for explicit user approval to proceed. 3. **State tracking: Begin every single response by declaring your current state, e.g. "

(1) Prompt + Công cụ

(2) Output AI

. - Option C: Isolated Boundaries + Happy-Path Interactions.

Pro : Light, hyper — fast execution. Con : Low risk protection for complex edge cases

. - PAUSE & ASK: Ask the user which strategy to apply. DO NOT PROCEED. </step> <step number="6" name="Test Case Generation & Selection Optimization"> Action: Synthesize the approved variables, partitions, boundaries, and combinatorial choices into final test cases. - Apply Selection Optimization Rules: - Valid Classes: Combine as many valid equivalence classes as possible into a single test case to minimize test bloat. - Invalid Classes: Isolate them. A test case must cover ONE and ONLY ONE invalid class at a time to prevent error masking. - Formatting Choice: Ask the user for their preferred output structure (Format A: CSV/Excel Matrix for automation; Format B: Markdown Documentation). - Final Output Generation: Generate the cases based on the choice. Every test case must include an "Assumptions Made" field documenting the specific structural decisions chosen during Steps 2-5. </step> </workflow> <initialization> To begin, acknowledge these exact core workflow state-machine rules, confirm your understanding of the strict "Stop and Wait" mechanics, state "

CurrentState : Initialization

", and prompt the user to provide the first functional requirement to launch Step 0. </initialization> ``` Please put the skill in the './.claude/skills' folder of the current workspace. Remember to use the format that Claude can detect it is a skill and can be use with a slash."

` . 4. **Ambiguity mitigation.** If you logical contradictions at any stage, with Pros /Cons for the user to select merge two steps into one response, Initialization When this skill is invoked workflow state-machine rules above mechanics. 3. State `

Cu:

` . 4. Prompt the user to provide the into Step 0 until the user has supplied requisite Validation & Elicitation Evaluation general (e.g., "test the login page") < **Identify gaps:** missing data types: clarifying questions:** suggest 3 to 5 mandatory fields, user roles, expected questions and wait. Do NOT proceed — Deconstruct Requirements (Variable scope of input and output data. - **named.** - **Identify implicit variables:** session state, timezones, global context table of all identified variables and time to confirm the variable list or input r Domain & Technical Constraints Definition **Apply categorization heuristics:** elements (e.g., BUS, TRUCK, TAXI) are identify mandatory conditions. - **Functional subjective constraints.** - **Strategic architectural maximum like Int32 maximum user to approve the domains or choose **Strategic selection options:** present — Pairwise/Orthogonal Array Testing interactions. - **Option B — Exhaust state explosion.** - **Option C — Isolated fast execution. Con : low risk protection which strategy to apply. Do NOT proceed Optimization Synthesize the approved into final test cases. - **Apply select equivalence classes as possible into isolate them. A test case must cover masking.** - **Formatting choice:** as CSV/Excel Matrix for automation; Format generation:** generate the cases based: "Assumptions Made" field documenting This is the final step — no further pause use r requests revisions. Created .cla Claude Code skill format (frontmatter testing in this project. Note: this skill here. To reuse it across all your projects (your user-level skills directory) — w****

Tool: Claude Code (Sonnet 5). Thời gian: 09:15 06/07/2026. **Prompt (rút gọn):** Gọi skill /domain-testing cho FR-04 (Quản lý hồ sơ cá nhân), kèm đặc tả tiếng Việt của giảng viên, yêu cầu AI tự đọc mã nguồn (Profile.jsx, server.js, database.js) trước khi đặt câu hỏi ở Step 0. Toàn văn: prompt_Log.txt dòng 6–774 (~10 lượt trao đổi qua đủ 7 bước).

AI tự phát hiện mâu thuẫn giữa regex và lỗi hổng leo thang quyền role qua "Assumption Legend" đầy đủ.

(1) Prompt + Công cụ	(2) Output AI
Tool: Claude Code (Sonnet 5). Thời gian: 13:20 06/07/2026. Prompt (rút gọn): Gọi /domain-testing cho FR-08 (Checkout), kèm đặc tả, yêu cầu AI tự kiểm tra Checkout.jsx, server.js, CartContext.jsx trước Step 0. <i>Toàn văn: prompt_Log.txt dòng 845–1120.</i>	Phát hiện 2 lỗi nghiêm trọng (tổng t giỏ hàng không bị xóa sau thanh to một lỗi.
Tool: Claude Code (Sonnet 5). Thời gian: 13:59 06/07/2026. Prompt (rút gọn): Gọi /domain-testing cho FR-19 (Quản lý người dùng — Admin), kèm đặc tả, yêu cầu AI đọc route admin/users trong server.js trước Step 0. <i>Toàn văn: prompt_Log.txt dòng 1124–1428.</i>	Phát hiện 2 lỗi hổng nghiêm trọng (k chống tự xóa tài khoản đang đăng r
Tool: Claude Code (Sonnet 5). Thời gian: 14:51 06/07/2026. Prompt (rút gọn): Gọi /domain-testing cho FR-11 (Lịch sử đơn hàng — Mobile), kèm đặc tả, yêu cầu AI so sánh GET /api/orders/my-orders với GET /api/orders/:id trước Step 0. <i>Toàn văn: prompt_Log.txt dòng 1431–1725.</i>	Phát hiện lỗi hổng IDOR nghiêm trọng hữu) và lỗi thiếu phân biệt màu trạn
Tool: Claude Code (Sonnet 5), phiên riêng để giữ prompt log sạch. Thời gian: gửi 16:12 06/07/2026. Prompt (rút gọn): Thực thi thật (curl) toàn bộ test case Direct-API của FR-04 trên backend vừa seed lại, cập nhật cột "Actual" bằng kết quả quan sát được thay vì dự đoán. <i>Toàn văn: prompt_Log.txt dòng 1728–1813.</i>	Chạy 32 test case Direct-API qua AP case "chưa xác định" (field-omission case".

(1) Prompt + Công cụ

(2) Output AI

Tool: Claude Code (Sonnet 5), cùng phiên thực thi. **Thời gian:** gửi 16:22 06/07/2026. **Prompt (rút gọn):** Tiếp tục thực thi Direct-API cho FR-08, dừng giỏ hàng cố định (58.000.000đ) trước khi chạy các case `total_amount`. *Toàn văn: `prompt_Log.txt` dòng 1813–1869.*

Chạy 19 test case; phát hiện thêm lỗi tổng tiền tăng gấp 10 lần (580.000.C là số nguyên thay vì phân số 0.10.

4. Tổng kết Độ chính xác AI

Tổng hợp verdict từ Mục 3 và điền vào bảng dưới.

Chi số	Số lượng	Tỉ lệ
Tổng artifact AI sinh đã audit	8	
VALID (đúng, dùng nguyên)	4	50%
INVALID (sai; loại bỏ)	0	0%
INCOMPLETE (chấp nhận sau khi sửa)	4	50%

5. Kết luận — Khi nào nên / không nên dùng AI?

Qua 4 tính năng, mẫu hình rõ nhất là: AI thực thi đúng và nhanh phương pháp luận EP/BVA (83 test case tổng cộng, đúng thuật toán biên LB/LB±1/UB/UB±1) nhưng luôn có xu hướng suy diễn thừa khi đặc tả mơ hồ (ví dụ gắn nhãn "thiếu route guard" là lỗi dù đặc tả chỉ yêu cầu chặn hành động thanh toán), và không thể biết trước hành vi runtime nếu không thực thi thật (lỗi coupon SAVE10, hành vi NULL-coercion chỉ lộ ra khi chạy code thật). Vì vậy AI mạnh nhất ở việc sinh bộ test case đầy đủ, có cấu trúc, và thực thi lặp lại ở quy mô lớn; AI yếu nhất ở việc tự quyết định "điều gì thực sự vi phạm đặc tả" khi văn bản không rõ ràng, và ở việc tự báo cáo số liệu tóm tắt (một phiên đã tự đếm sai 27/32 test case đã chạy). Khuyến nghị: nên dùng AI cho giai đoạn thiết kế test case và thực thi số lượng lớn, nhưng mọi phát hiện lỗi phải được người kiểm thử xác minh độc lập với đặc tả gốc và hành vi hệ thống thật trước khi đưa vào báo cáo chính thức - không nên dùng AI như "trọng tài" cuối cùng cho việc gì là lỗi.

6. Mandatory Disclosure (dán nguyên văn)

"Bộ test case Domain Testing/BVA, Main Report, và bản nháp GitHub Issues cho 4 tính năng FR-04/FR-08/FR-19/FR-11 được sinh phiên bản đầu bởi Claude Code (Claude Sonnet 5, Claude Opus 4.8); tôi đã rà soát và chỉnh sửa các phát hiện bị suy diễn sai hoặc gắn nhãn không nhất quán (lỗi "thiếu route guard /checkout" ở FR-08, nhãn TC-30 ở FR-04, bảng biến V3/V4 bị lỗi định dạng ở FR-19), bổ sung việc loại trừ 1 phát hiện phụ ngoài phạm vi (lỗi coupon SAVE10, thuộc FR-09) khỏi danh sách lỗi chính thức; toàn bộ kiểm thử thủ công qua giao diện Web/Mobile/Admin, ảnh chụp màn hình bằng chứng, việc tạo 14 GitHub Issues thật trên repo nhóm, và đối chiếu chéo từng issue với báo cáo do tôi tự thực hiện. AI Audit Report chi tiết đính kèm ở Phụ lục A. Tôi cam đoan không dùng AI để sinh bất kỳ artifact nào thuộc danh mục bị cấm."

Chữ ký

Họ tên sinh viên (in hoa):	LÊ THIÊN PHÚ
MSSV:	23127244
Lớp / Khoá:	23KTPM2
Môn học:	CS423 / CSC13003 – Kiểm chứng Phần mềm
Giảng viên:	Dr. Lâm Quang Vũ
Ngày:	03/07/2026

Chữ ký: 

Tham khảo

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.